SQL Triggers

RGC UPEKSHA

What is a SQL Trigger?

- An SQL trigger is a special type of stored procedure in a database that automatically executes a set of pre-defined SQL statements when a specific event occurs. These events are typically related to changes in data within a particular table. Think of them as automated mini-programs that react to certain database activities.

Types of triggers

- **DML triggers**: These react to data manipulation language (DML) events like INSERT, UPDATE, and DELETE on a specific table. For example, a trigger could automatically update a related table whenever a new row is inserted into another table.
- **DDL triggers**: These respond to data definition language (DDL) events like CREATE, ALTER, and DROP on database objects like tables or views.
- **Logon triggers**: These are triggered by user logon events, allowing you to implement specific actions or restrictions based on who is accessing the database.

Benefits

- **Enforcing data integrity**: You can use them to validate data before it's inserted or updated, ensuring it adheres to specific rules.
- **Automating tasks**: They can automate repetitive tasks triggered by data changes, saving time and effort.
- **Maintaining consistency**: Triggers can help maintain consistency across different parts of your database by automatically updating related data.

DML triggers

- BEFORE Triggers – This type of trigger fires before the data has been committed into the database.
- AFTER Triggers – This type of trigger fires after the event it is associated with completes and can only be defined on permanent tables.
- INSTEAD OF Triggers – This type of trigger fires instead of the event it is associated with and can be applied to tables or views.

Event, Condition, and Action in SQL Triggers

- An SQL trigger follows the Event-Condition-Action (ECA) model

1. Event: This is the specific action that triggers the execution of the trigger. It's usually a DML (Data Manipulation Language) event like:

INSERT: A new row is added to a table.

UPDATE: An existing row in a table is modified.

DELETE: A row is removed from a table.

Less common are DDL (Data Definition Language) events like creating, altering, or dropping tables or views. Some databases also support logon triggers that fire based on user login events.

Event, Condition, and Action in SQL Triggers

- An SQL trigger follows the Event-Condition-Action (ECA) model

2. Condition (Optional): This is an optional clause that specifies further criteria for when the trigger's action should be executed. It's typically written as a WHERE clause and evaluates to True or False for each affected row. If no condition is specified, the action always executes after the event.

Event, Condition, and Action in SQL Triggers

- An SQL trigger follows the Event-Condition-Action (ECA) model

3. Action: This is the set of SQL statements that are executed when the trigger fires. It can include:

DML statements: Inserting, updating, or deleting data in other tables.

DCL statements: Granting or revoking permissions.

SELECT statements: Retrieving data to perform calculations or validations.

Calling stored procedures or functions: Executing complex logic or accessing external resources.

Example

```
CREATE TRIGGER update_total_sales
AFTER INSERT ON sales
FOR EACH ROW
WHEN NEW.sale_amount > 100 -- Optional condition
BEGIN
  UPDATE summary
  SET total_sales = total_sales + NEW.sale_amount
  WHERE product_id = NEW.product_id;
END;
```

Event: The trigger fires after an INSERT operation on the "sales" table.

Condition (Optional): This trigger only updates the "total_sales" if the new sale amount is greater than 100.

Action: The trigger updates the "total_sales" column in the "summary" table for the product associated with the new sale.

Basic Syntax for a DML Trigger

```
CREATE TRIGGER [schema_name.]trigger_name -- Create Trigger Statement
ON { table_name | view_name }
{ FOR | AFTER | INSTEAD OF } {[INSERT],[UPDATE],[DELETE]}
[NOT FOR REPLICATION]
AS
    {sql_statements}
```

INSERTED and DELETED Table Reference

In SQL triggers, INSERTED and DELETED are virtual tables specifically available within the trigger code. They act as temporary representations of the data affected by the triggering event:

INSERTED: Contains the data being inserted into the table if the trigger is triggered by an INSERT operation.

DELETED: Contains the data being deleted from the table if the trigger is triggered by an UPDATE or DELETE operation.

The INSERT Trigger

First let's create a Log table called "Records".

```
create table Records (
        EmpID int,
        EmpName Varchar (20),
        Status varchar (10);
        UpdatedOn datetime,
);
```

The INSERT Trigger

let's create a simple INSERT trigger to capture any INSERT actions that a user might apply to the Employee table

```
create trigger empInsertRecords
on Employee
for insert
as
        insert into Records(EmpID,EmpName,UpdatedOn)
        select eid,ename,'inserted' ,GETDATE()
        from inserted
go
```

The DELETE Trigger

let's create a simple Delete trigger to capture any delete actions that a user might apply to the Employee table

```
create trigger empDeleteRecord
on Employee
for delete
as
        insert into Records(EmpID,EmpName,Status,UpdatedOn)
        select eid,ename,'deleted',GETDATE()
        from deleted
go
```

The UPDATE Trigger

let's create a simple Update trigger to capture any update actions that a user might apply to the Employee table

```
create trigger empUpdateRecord
on Employee
for update
as
        insert into Records(EmpID,EmpName,Status,UpdatedOn)
        select eid,ename,'updated',GETDATE()
        from inserted
go
```

INSTEAD OF Clause

In SQL, INSTEAD OF triggers are a specific type of trigger used for managing operations on views. Unlike traditional triggers that react to changes in tables, these triggers intercept and replace INSERT, UPDATE, or DELETE operations on a view with alternative actions.

Purpose:

They allow you to manipulate data indirectly through a view, even if the view itself wouldn't normally support modifications.

They're useful for enforcing specific validation or logic when modifying data accessed through the view.

Scope:

They can only be defined on views, not directly on tables.

They can be created for one specific operation (INSERT, UPDATE, or DELETE) on the view.

INSTEAD OF Clause

let's create a view for the users to access instead of the table itself

```
create view Managers
as
        select
                eid,
                ename,
                did
                from Employee, Dept
                where eid=managerid
go
```

INSTEAD OF Clause

Now, let's create the INSTEAD OF trigger on Managers table. We do not allow a user to delete any rows in this trigger. Instead the department id will be removed in the view.

```
create trigger ManagerDoNotDelete
on Managers
instead of delete
as
        update Managers
        set did = ' '
        where eid in (select eid from deleted)
go
```

AFTER versus FOR

Both refer to triggers that execute a set of SQL statements after a specific event occurs on a table, such as INSERT, UPDATE, or DELETE.

The Before Trigger

While there is the concept of before triggers in some specific contexts, it's generally not a standard term used in most widely used SQL implementations.

1. Trigger Timing:

Most common SQL databases only offer AFTER triggers that execute after the specific event (INSERT, UPDATE, DELETE) has already occurred on the table. They don't directly have "before" triggers that fire before the event happens.

2. DDL vs DML triggers:

Some databases might have triggers for Data Definition Language (DDL) events like CREATE, ALTER, or DROP on tables or views. These events happen before the actual change is applied to the database schema. However, these are not typically called "before triggers" and have separate mechanisms.